

Práctica #1: Documento Técnico
Fundamentos de Eficiencia Computacional y Tipos de Datos
Abstractos
Estructura Lineal: Arreglos Dinámicos

Joanfer Hidalgo Chaves

Escuela de Informática y Computación

Universidad Nacional de Costa Rica

EIF207, Estructuras de Datos

Msc. Johnny Villalobos Murillo

II Ciclo 2026

8 de agosto de 2026

Recursos

Este documento también forma parte del repositorio del proyecto, junto con el código fuente completo y su documentación generada con Doxygen:

Repositorio de GitHub: https://github.com/Cetrei/estructuras_practical1
Documentación en línea (GitHub Pages): https://Cetrei.github.io/estructuras_practical1/

Índice

1. Descripción de las Estructuras Utilizadas	3
1.1. Estructura Estudiante	3
1.2. Estructura Historial	3
1.3. Estructura Estadísticas	3
1.4. Estructura CodigoResultado	3
2. Algoritmo para Determinar la Cantidad de Registros	4
3. Algoritmo para Cargar el Arreglo Dinámico	4
4. Algoritmo para Calcular Nota Menor, Nota Mayor y Nota Promedio	5
5. Algoritmo para Localizar al Estudiante de Nota Menor y Nota Mayor	5
6. Análisis del Costo Computacional	6
6.1. Supuestos	6
6.2. contarRegistros()	6
6.3. cargarHistorial()	6
6.4. calcularEstadísticas()	7
6.5. buscarEstudiante()	7
6.6. Costo Total del Programa	7
6.7. Tabla Resumen	8

1 Descripción de las Estructuras Utilizadas

El programa define cuatro estructuras (`struct` y `enum`) para representar la información del ejercicio. Todos los tamaños de cada atributo son fijos para poder leer y escribir los registros como se solicita sin generar errores.

1.1 Estructura Estudiante

Representa un registro del archivo `estudiantes.dat`.

```
1 struct Estudiante:
2     carnet[15]
3     nombre[40]
4     apellidos[40]
5     carrera[40]
6     nivel
```

1.2 Estructura Historial

Representa un registro del archivo `historial.dat`.

```
1 struct Historial:
2     carnet[15]
3     codigoCurso[10]
4     nombreCurso[50]
5     ciclo
6     anio
7     nota
```

1.3 Estructura Estadísticas

Agrupar los resultados solicitados respecto a los datos almacenados en el historial.

```
1 struct Estadisticas:
2     notaMenor
3     notaMayor
4     notaPromedio
5     carnetNotaMenor[15]
6     carnetNotaMayor[15]
```

1.4 Estructura CodigoResultado

Representa el estado de salida de una operación que puede fallar de más de una forma.

```
1 enum CodigoResultado:
2     EXITO
3     ARCHIVO_NO_ENCONTRADO
4     MEMORIA_INSUFICIENTE
```

2 Algoritmo para Determinar la Cantidad de Registros

Función: `contarRegistros(const char* nombreArchivo, int* cantidadRegistros)`

El algoritmo evita leer el contenido del archivo para saber cuántos registros contiene. En su lugar, se apoya en el tamaño del archivo en bytes:

1. Abrir el archivo en modo de lectura ("rb").
2. Si el archivo no existe o no se pudo abrir, retornar `ARCHIVO_NO_ENCONTRADO`
3. Mover el cursor de lectura al final del archivo
4. Obtener la posición del cursor (el tamaño total en bytes)
5. Cerrar el archivo
6. Dividir el tamaño en bytes entre `sizeof(Historial)` para obtener la cantidad de registros completos almacenados

3 Algoritmo para Cargar el Arreglo Dinámico

Función: `cargarHistorial(const char* nombreArchivo, int cantidadRegistros, Historial** historiales)`

1. Abrir el archivo en modo de lectura
2. Si no se pudo abrir, retornar `ARCHIVO_NO_ENCONTRADO`
3. Reservar memoria dinámica para `cantidadRegistros` elementos de tipo `Historial`
4. Si la reserva de memoria falla, cerrar el archivo y retornar `MEMORIA_INSUFICIENTE`
5. Leer todos los registros del archivo hacia el arreglo reservado indicando el tamaño de cada elemento y la cantidad total a leer.
6. Cerrar el archivo y retornar `EXITO`

La carga se realiza en una sola lectura en vez de una por registro ya que recibe la cantidad de elementos a leer y los copia seguido en el arreglo aprovechando que el struct en memoria y archivo comparten el mismo tamaño por campo.

4 Algoritmo para Calcular Nota Menor, Nota Mayor y Nota Promedio

Función: `calcularEstadisticas(const Historial historiales[], int cantidadRe`

En vez de recorrer el arreglo tres veces (una para el mínimo, otra para el máximo y otra para el promedio), el algoritmo realiza un único recorrido que actualiza los tres resultados simultáneamente.

1. Inicializar `notaMenor` y `notaMayor` con la nota del primer registro, y sus carnets asociados con el carnet del primer registro
2. Para cada registro del arreglo:
 - a) Sumar su nota a un contador `sumaNotas`.
 - b) Si su nota es menor que `notaMenor`, actualizar `notaMenor` y `carnetNotaMenor`
 - c) Si su nota es mayor que `notaMayor`, actualizar `notaMayor` y `carnetNotaMayor`
3. Calcular `notaPromedio` dividiendo `sumaNotas` entre la cantidad de registros

5 Algoritmo para Localizar al Estudiante de Nota Menor y Nota Mayor

Función: `buscarEstudiante(const char* nombreArchivo, const char* carnet, Estudiante* estudianteEncontrado)`

1. Conseguir el carnet de la nota menor y el carnet de la nota mayor con `calcularEstadisticas()`
2. Iniciar la búsqueda para el carnet de la nota menor o mayor
3. Abrir el archivo de estudiantes en modo de lectura.
4. Si no se pudo abrir, retornar `ARCHIVO_NO_ENCONTRADO`
5. Leer registros uno a uno sin cargar el archivo completo en memoria
6. Comparar el carnet de cada registro leído contra el carnet buscado
7. Si coinciden, copiar el registro encontrado en el parámetro de salida, cerrar el archivo y retornar `EXITO`
8. Si se llega al final del archivo sin encontrar coincidencia, cerrar el archivo y retornar `NO_ENCONTRADO`

Se debe llamar esta función dos veces; una con el carnet de la nota menor y otra con el carnet de la nota mayor.

6 Análisis del Costo Computacional

6.1 Supuestos

Para el análisis se consideran las siguientes variables:

- n = cantidad de registros almacenados en `historial.dat`.
- m = cantidad de registros almacenados en `estudiantes.dat`.

Para cada función se presenta el razonamiento que deduce su costo, distinguiendo:

- **Complejidad algorítmica:** la cota asintótica reducida
- **Complejidad real:** el conteo efectivo de operaciones, sin simplificar, incluyendo constantes y pasos fijos
- **Complejidad espacial:** la memoria adicional reservada por la función, en función de la cantidad de estructuras de datos auxiliares que usa

6.2 contarRegistros()

La función realiza una cantidad fija de operaciones sin importar el tamaño del archivo:

- `fopen`: abrir el archivo.
- `fseek`: mover el cursor al final del archivo.
- `ftell`: leer la posición del cursor.
- `fclose`: cerrar el archivo.

Ninguna de estas operaciones depende de n , porque `fseek` y `ftell` consultan el tamaño del archivo, no lo recorren

Complejidad real: $O(4)$. 4 operaciones de E/S (`fopen`, `fseek`, `ftell`, `fclose`) Complejidad algorítmica: $O(1)$. Ya que el costo no depende de n Complejidad espacial: $E(1)$. La función solo reserva variables locales de tamaño fijo.

6.3 cargarHistorial()

La función abre el archivo, reserva memoria para `cantidadRegistros` elementos, y realiza una única llamada que transfiere los n registros del archivo al arreglo. La lectura en bloque copia los n registros de forma continua, por lo que internamente crece proporcionalmente a n : cada uno de los n registros se copia una vez.

Complejidad real: $O(n + 3)$. 1 apertura, 1 reserva de memoria, n registros transferidos, 1 cierre Complejidad algorítmica: $O(n)$. El costo dominante es la transferencia de los n registros Complejidad espacial: $E(n)$. Se reserva un arreglo de n elementos de tipo `Historial`, cuyo tamaño crece proporcionalmente a la cantidad de registros

6.4 calcularEstadisticas()

La función recorre el arreglo de n historiales una única vez. En cada iteración realiza una cantidad fija de operaciones:

- sumar la nota al contador de suma
- comparar contra la nota menor
- comparar contra la nota mayor
- y en el peor caso copiar el carnet en ambas comparaciones

Como estas comparaciones y copias no dependen de n individualmente (son de tamaño fijo, 15 caracteres), el costo por iteración es constante, y se repite n veces.

Complejidad real: $O(n)$. Al ser una iteración lineal sobre n elementos Complejidad algorítmica: $O(n)$ Complejidad espacial: $E(1)$. La función solo usa variables locales de tamaño fijo

6.5 buscarEstudiante()

La función recorre el archivo de estudiantes registro por registro, comparando el carnet de cada uno contra el carnet buscado, hasta encontrar coincidencia o llegar al final. En el peor caso (el carnet buscado es el último registro, o no existe), se leen y comparan los m registros del archivo.

Complejidad real: $O(m + 2)$. 1 apertura, hasta m lecturas (`fread`) en el peor caso (no se encuentra o el registro buscado es el último), 1 cierre Complejidad algorítmica: $O(m)$ Complejidad espacial: $E(1)$. La función solo mantiene un registro `Estudiante` a la vez en memoria

Esta función se invoca dos veces desde el flujo principal; una por el carnet de la nota menor y otra por el de la nota mayor. Por lo que el costo real de las dos búsquedas combinadas es $O(m + 2) + O(m + 2) = O(2m + 4)$

6.6 Costo Total del Programa

Sumando el costo real de cada operación ejecutada desde el flujo principal (usando el desglose real de cada función, sin simplificar), incluyendo las operaciones de costo constante que arman e imprimen el reporte y la liberación final de memoria:

$$O(4) + O(n + 3) + O(n) + O(m + 2) + O(m + 2) + O(1) + O(1) + O(1)$$

Agrupando términos semejantes:

$$O(2n + 2m + 14)$$

Complejidad real: $O(2n + 2m + 14)$ Complejidad algorítmica: $O(n + m)$. Al eliminar constantes y factores menores Complejidad espacial: $E(n)$. Está dominada por el arreglo dinámico de historiales reservado en `cargarHistorial()`, ya que el resto de las funciones usa memoria constante.

6.7 Tabla Resumen

Función	Complejidad algorítmica	Complejidad espacial
<code>contarRegistros()</code>	$O(1)$	$E(1)$
<code>cargarHistorial()</code>	$O(n)$	$E(n)$
<code>calcularEstadisticas()</code>	$O(n)$	$E(1)$
<code>buscarEstudiante()</code> (nota menor)	$O(m)$	$E(1)$
<code>buscarEstudiante()</code> (nota mayor)	$O(m)$	$E(1)$
<code>construirReporte()</code>	$O(1)$	$E(1)$
<code>imprimirReporte()</code>	$O(1)$	$E(1)$
Programa completo	$O(n + m)$	$E(n)$